

5 - Advanced Clustering Techniques

Giovanni Della Lunga
giovanni.dellalunga@unibo.it

Advanced Topics in Artificial Intelligence

Bologna - March, 11-28 2025

Spectral Clustering

Subsection 1

Introduction

What is Spectral Clustering?

Introduction

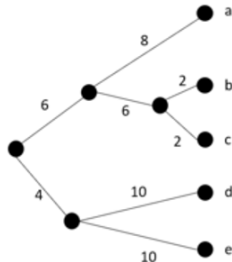
- A graph-based clustering method that uses eigenvalues of similarity matrices to find clusters.
- Unlike k-means, it captures non-convex and arbitrarily shaped clusters.
- Particularly effective when clusters have non-linear boundaries.
- Applications: Image segmentation, social network analysis, bioinformatics, recommendation systems.
- In the next slides, we briefly introduce the individual steps of the procedure and then move on to an in-depth analysis of each one...

Spectral Clustering Step by Step

Introduction

1. Similarity Matrix Creation: The first step involves creating a similarity matrix that *represents how similar each pair of points in the dataset is to each other*. This is often done using the Gaussian (RBF) kernel to convert the Euclidean distances between data points into similarity scores.

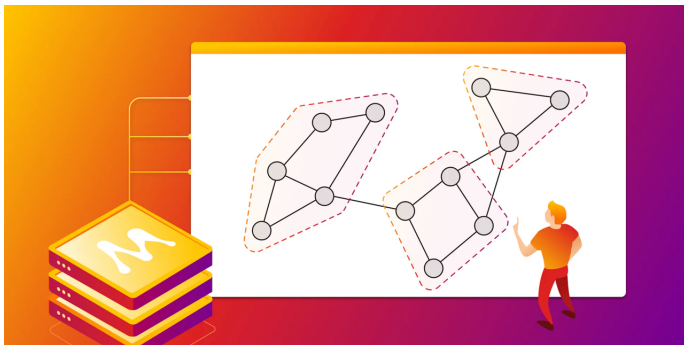
M	a	b	c	d	e
a	0	16	16	28	28
b	16	0	4	28	28
c	16	4	0	28	28
d	28	28	28	0	20
e	28	28	28	20	0



Spectral Clustering Step by Step

Introduction

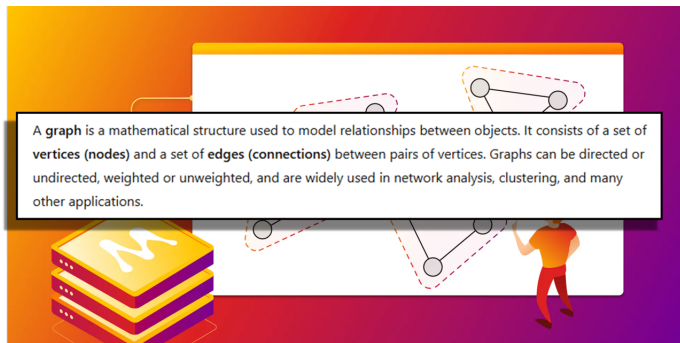
2. Graph Representation: The similarity matrix is then used to represent the data as a graph, with data points as nodes and the similarities as weights on the edges between nodes.



Spectral Clustering Step by Step

Introduction

2. Graph Representation: The similarity matrix is then used to represent the data as a graph, with data points as nodes and the similarities as weights on the edges between nodes.



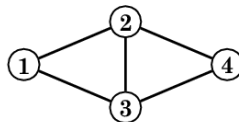
Spectral Clustering Step by Step

Introduction

3. Laplacian Matrix: From this graph, a Laplacian matrix is computed. The Laplacian is a matrix representation that captures the structure of the graph. It is used to find the minimum number of cuts needed to partition the graph into clusters.

$$D = \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 3 & 0 \\ 0 & 0 & 0 & 2 \end{bmatrix}$$

$$A = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix}$$



$$L = \begin{bmatrix} 2 & -1 & -1 & 0 \\ -1 & 3 & -1 & -1 \\ -1 & -1 & 3 & -1 \\ 0 & -1 & -1 & 2 \end{bmatrix}$$

Spectral Clustering Step by Step

Introduction

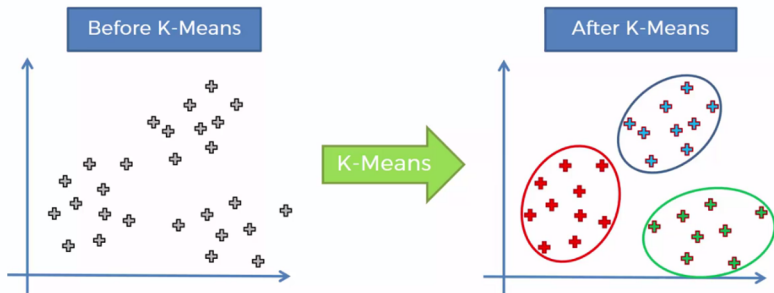
4. Eigenvalue Decomposition: The next step involves computing the eigenvalues and eigenvectors of the Laplacian matrix. The eigenvectors corresponding to the **smallest** eigenvalues (except for the smallest one, which is always zero) are used. These eigenvectors are then stacked to form a new set of features with reduced dimensions.

The diagram illustrates the eigenvalue decomposition equation $Ax = \lambda x$. The matrix A is green, and the vector x is red. The eigenvalue λ is blue, and the vector x is red. A green arrow points from the text " $n \times n$ Matrix" to the matrix A . A red arrow points from the text "Eigenvector" to the vector x on the left. A blue arrow points from the text "Eigenvalue" to the eigenvalue λ . Above the matrix A and the eigenvalue λ , there are red arrows pointing to the right, indicating the direction of the vectors.

Spectral Clustering Step by Step

Introduction

5. Clustering: The reduced dataset is then clustered using a conventional algorithm like k-means. Since the data is now in a space where clusters are more distinguishable, k-means or similar algorithms can effectively identify the clusters.



Subsection 2

Mathematical Background

What is a Similarity Matrix?

Mathematical Background

The first step in spectral clustering is crucial as it lays the foundation for the entire clustering process. Let's delve into the details of this step:

- A **similarity matrix**, sometimes also referred to as an affinity matrix, is a *square matrix* used to represent the similarity between each pair of points in a dataset.
- In the context of spectral clustering, the dataset is typically composed of multidimensional points, and **the goal is to measure how close or similar these points are to one another**.
- The similarity matrix is **symmetric**, with its diagonal elements representing the similarity of each point with itself (usually the maximum similarity score) and off-diagonal elements representing the similarity between different points.

How is the Similarity Matrix Created?

Mathematical Background

- **Calculating Pairwise Distances:** The process begins by calculating the pairwise distances between all points in the dataset. This is often done using Euclidean distance, which measures the straight-line distance between two points in Euclidean space. However, other distance metrics can also be used depending on the nature of the data and the specific requirements of the clustering task.
- **Converting Distances to Similarities:** Directly using distances as measures of similarity is not intuitive because in a similarity context, we expect similar points to have a high score. Hence, the distances need to be converted into similarity scores. This conversion is typically achieved using a similarity function, such as the Gaussian (Radial Basis Function, RBF) kernel.

Using the Gaussian (RBF) Kernel

Mathematical Background

- The Gaussian kernel is a popular choice for transforming Euclidean distances into similarity scores. It is defined as follows:

$$S(x, y) = \exp\left(-\frac{\|x - y\|^2}{2\sigma^2}\right)$$

Here, $S(x, y)$ is the similarity between points x and y , $\|x - y\|$ is the Euclidean distance between x and y , and σ is a parameter that controls the width of the neighborhood or the spread of the Gaussian kernel.

- The effect of this transformation is that points closer to each other (smaller distances) result in higher similarity scores (closer to 1), while points further apart have lower scores (closer to 0).

Key Considerations

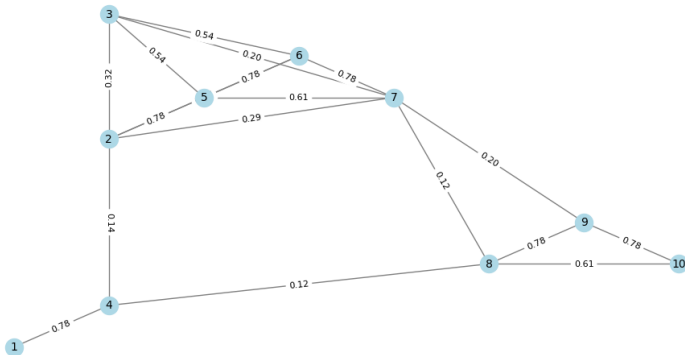
Mathematical Background

- **Choice of σ :** The parameter σ plays a crucial role in determining the scale of similarity. Choosing an appropriate σ is essential for the success of the clustering.
 - A **small** σ makes the similarity drop off quickly with distance, leading to a sparser similarity matrix where only very close points are considered similar.
 - A **larger** σ makes the similarity drop off more slowly, considering a broader range of points as similar.
- **Sparsity of the Matrix:** In practice, for large datasets, the similarity matrix can be made sparse by setting all similarity scores below a certain threshold to zero. This reduces the computational complexity and focuses on stronger relationships, assuming that very distant points (low similarity) do not contribute significantly to the clustering structure.

Transform a Dataset into a Graph

Mathematical Background

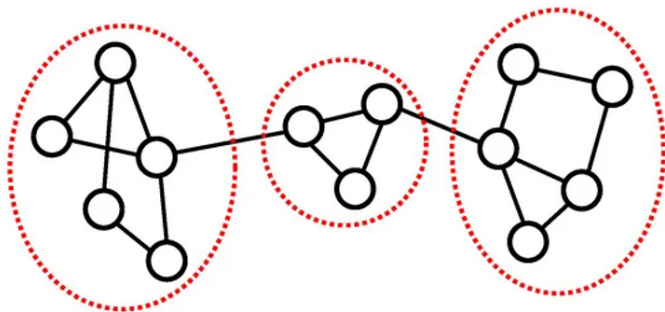
Graph Representation of the Dataset with Similarity Weights



See the Jupyter Notebook **chapter-05-01.ipynb** for details and code

Transform a Dataset into a Graph

Mathematical Background



The intuition is that cutting the graph into subgraphs (clusters) can be done in such a way that **the connections (edges) between different clusters are weak (low similarity)**, while connections within a cluster are strong (high similarity)

Graph Laplacians

Mathematical Background

A quick recap of what we've done so far:

- Define similarity matrix $A \in \mathbb{R}^{n \times n}$ where A_{ij} represents the similarity between points i and j .
- Represent data as an undirected weighted graph $G = (V, E)$, where V is the set of data points and E represents weighted edges.
- The degree of each node is given by $D_{ii} = \sum_j A_{ij}$ (**sum of the weights of all edges connected to node i**), forming the diagonal degree matrix D .

Graph Laplacians

Mathematical Background

The **Laplacian matrix** is a crucial concept in graph theory and spectral clustering, in a nutshell it is a matrix representation that captures the graph's structure, focusing on how nodes (data points) are connected to each other via edges (similarities).

- The **unnormalized Laplacian** is defined as:

$$L = D - A \quad (1)$$

- The **normalized Laplacians** are given by:

$$L_{sym} = D^{-1/2} L D^{-1/2} = I - D^{-1/2} A D^{-1/2} \quad (2)$$

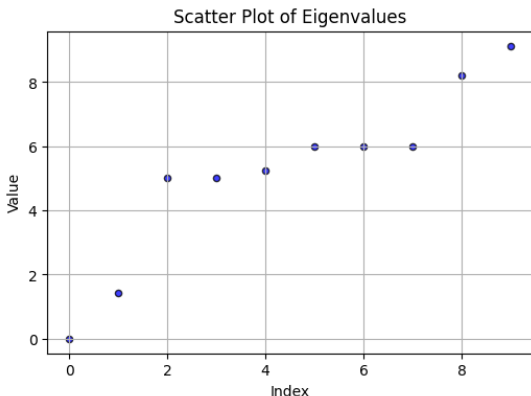
$$L_{rw} = D^{-1} L = I - D^{-1} A \quad (3)$$

- The spectrum (eigenvalues) of L reveals the structure of clusters .

Interpreting the Plot of Eigenvalues of the Laplacian

Mathematical Background

As we have said, the **eigenvalues of the Laplacian matrix** provide insights into the **structure** of a graph, including connectivity, clustering, and diffusion properties. When we plot them, we can extract valuable information about the dataset.



Interpreting the Plot of Eigenvalues of the Laplacian

Mathematical Background

1. Eigenvalues and Graph Connectivity

- The smallest eigenvalue (λ_1) of the Laplacian **is always 0** for a connected graph.
- The number of **zero eigenvalues** tells us how many **disconnected components** exist in the graph.
- If **only one** eigenvalue is 0 $\rightarrow$ The graph is **fully connected**.
- If there are **k zero eigenvalues** $\rightarrow$ The graph consists of **k disjoint clusters**.

Interpreting the Plot of Eigenvalues of the Laplacian

Mathematical Background

2. Spectral Gap and Cluster Structure

- The **second-smallest eigenvalue** (λ_2 , the *Fiedler eigenvalue*) determines how **easily** the graph can be split into two clusters
 - **Small** $\lambda_2 \rightarrow$ Weakly connected graph $\rightarrow$ Easier to separate into two groups
 - **Large** $\lambda_2 \rightarrow$ Strongly connected graph $\rightarrow$ Harder to split.
- The **gaps between consecutive eigenvalues** help determine the **optimal number of clusters**.
- A **large jump** between λ_k and λ_{k+1} suggests that **k clusters** are a natural choice.

Choosing the Number of Clusters

- Analyze the **eigenvalue gap** (the spectral gap heuristic).
- Plot the sorted eigenvalues of L and locate the largest gap.
- The optimal k is determined by the index where the gap is largest.

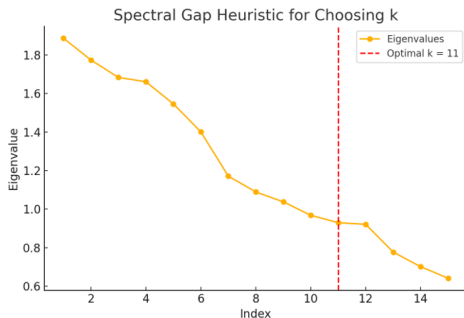


Figure: Spectral gap heuristic for determining k .

The Meaning of Eigenvectors in Spectral Clustering

Mathematical Background

Eigenvectors as Coordinates in a New Feature Space

- The original dataset is mapped into a new space defined by the eigenvectors of the Laplacian matrix.
- The first few non-trivial eigenvectors contain information about how nodes (data points) are connected.
- Clusters become more separable in this new space.

Spectral Clustering Algorithm

Algorithm 1 Spectral Clustering Algorithm

- 1: Construct the similarity graph and compute its adjacency matrix A .
 - 2: Compute the degree matrix D .
 - 3: Compute the graph Laplacian matrix L .
 - 4: Compute the first k eigenvectors of L to form an embedding matrix U .
 - 5: Normalize each row of U (for normalized Laplacian).
 - 6: Apply k-means clustering to the rows of U .
 - 7: Assign each point to a cluster based on k-means output.
-

Subsection 3

Comparison with Other Methods

Spectral Clustering vs. k-Means

Comparison with Other Methods

- k-means assumes convex clusters and uses Euclidean distance, which fails for complex structures.
- Spectral clustering leverages graph-based structures and does not assume convexity.
- Spectral clustering is computationally more expensive due to eigen decomposition ($O(n^3)$ complexity).

Spectral Clustering vs. Hierarchical Clustering

Comparison with Other Methods

- Spectral clustering is based on eigenvectors, whereas hierarchical methods use linkage criteria.
- Hierarchical clustering produces a dendrogram without requiring k upfront.
- Spectral clustering is typically more robust for high-dimensional data.

Spectral Clustering vs. Hierarchical Clustering

Comparison with Other Methods

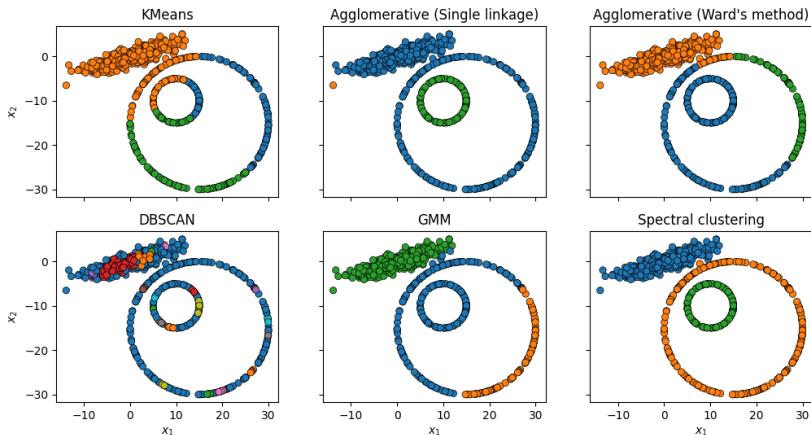


Figure: See the jupyter notebook **chapter-05-01.ipynb** for explanations

Limitations of Spectral Clustering

Comparison with Other Methods

- High computational cost due to eigen decomposition ($O(n^3)$ for large n).
- Performance depends heavily on the choice of similarity measure.
- Requires careful tuning of the similarity graph construction.

Conclusion

Summary

- Spectral clustering leverages graph theory and eigenvalues to improve clustering.
- It captures complex, non-linear cluster structures better than traditional methods.
- While computationally expensive, it is useful for a variety of applications where traditional methods fail.